Appendix A: F# Grammar Summary

This appendix summarizes the grammar of the F# language. The following table describes the notation conventions used in the grammar.

Notation Conventions in Grammar Rules

Notation	Description	Example
element-name$_{opt}$	The $_{opt}$ subscript indicates that element-name is optional.	**let rec**$_{opt}$
...	An ellipsis indicates that the preceding non-terminal construct and the separator token can repeat any number of times.	*expr* ',' ... ',' *expr*
keyword	Boldface type identifies a language keyword that must appear verbatim.	**module** *long-ident module-elems*
element-name	Italics identify an element that is defined in the grammar.	*script-fragment* : *module-elems*
[char1 – char2]	All ASCII characters in the range from *char1* to *char2*, inclusive.	[a – z]
[^ char1 – char2]	All ASCI characters except those in the specified range.	[^ A – Z]
'symbol' or "symbol"	The literal *symbol* is used in the grammar.	'(', "if"
(spec)	Parentheses enclose required individual grammar elements.	(+|-)
$token	Lexical analysis inserts *$token* as a hidden symbol.	$app

A.1 Lexical Grammar

A.1.1 Whitespace

```
whitespace : ' '+

newline :
      '\n'
      '\r' '\n'

whitespace-or-newline :
      whitespace
      newline
```

A.1.2 Comments

```
block-comment-start : "(*"
```

```
block-comment-end : "*)"

end-of-line-comment : "//" [^'\n' '\r']*
```

A.1.3 Conditional Compilation

```
    if-directive : "#if" whitespace ident-text

    else-directive : "#else"

    endif-directive : "#endif"
```

A.1.4 Identifiers and Keywords

A.1.4.1 Identifiers

```
digit-char : [0-9]

letter-char :
      '\Lu'
      '\Ll'
      '\Lt'
      '\Lm'
      '\Lo'
      '\Nl'
connecting-char : '\Pc'

combining-char :
      '\Mn'
      '\Mc'

formatting-char : '\Cf'

ident-start-char :
      letter-char
      _

ident-char :
      letter-char
      digit-char
      connecting-char
      combining-char
      formatting-char
      '
      _

ident-text : ident-start-char ident-char*

ident :
      ident-text
      `` ( [^'`' '\n' '\r' '\t'] | '`' [^ '`' '\n' '\r' '\t'] )+ ``
```

A.1.4.2 Long Identifiers

```
long-ident :  ident '.' ... '.' ident
long-ident-or-op :
      long-ident '.' ident-or-op
      ident-or-op
```

A.1.4.3 Keywords

```
ident-keyword : one of
```

```
      abstract and as assert base begin class default delegate do done
      downcast downto elif else end exception extern false finally for
      fun function global if in inherit inline interface internal lazy let
      match member module mutable namespace new null of open or
      override private public rec return sig static struct then to
      true try type upcast use val void when while with yield

reserved-ident-keyword : one of
      atomic break checked component const constraint constructor
      continue eager fixed fori functor include
      measure method mixin object parallel params process protected pure
      recursive sealed tailcall trait virtual volatile

reserved-ident-formats :
      ident-text ( '!' | '#')
```

A.1.4.4 Symbolic Keywords

```
symbolic-keyword : one of
      let! use! do! yield! return!
      | -> <- . : ( ) [ ] [< >] [| |] { }
      ' # :?> :? :> .. :: := ;; ; =
      _ ? ?? (*) <@ @> <@@ @@>

reserved-symbolic-sequence :
      ~ `
```

A.1.5 Strings and Characters

```
escape-char :  '\' ["\'ntbr]

non-escape-chars :  '\' [^"\'ntbr]

simple-char-char : any char except
      '\n' '\t' '\r' '\b' ' \ "

unicodegraph-short : '\' 'u' hexdigit hexdigit hexdigit hexdigit

unicodegraph-long :  '\' 'U' hexdigit hexdigit hexdigit hexdigit
                             hexdigit hexdigit hexdigit hexdigit

char-char :
      simple-char-char
      escape-char
      trigraph
      unicodegraph-short

string-char :
      simple-string-char
      escape-char
      non-escape-chars
      trigraph
      unicodegraph-short
      unicodegraph-long
      newline

string-elem :
      string-char
      '\' newline whitespace* string-elem

char : ' char-char '
```

```
string : " string-char* "

verbatim-string-char :
      simple-string-char
      non-escape-chars
      newline
      \
      ""

verbatim-string : @" verbatim-string-char* "

bytechar           :  ' simple-or-escape-char 'B

bytearray          :  " string-char* "B

verbatim-bytearray : @" verbatim-string-char* "B

simple-or-escape-char :
      escape-char
      simple-char

simple-char : any char except
      newline, return, tab, backspace,',\,"

triple-quoted-string : """ simple-or-escape-char* """
```

A.1.6 Numeric Literals

```
digit  : [0-9]

hexdigit     :
      digit
      [A-F]
      [a-f]

octaldigit : [0-7]

bitdigit : [0-1]

int : digit+

xint :
      0 (x|X) hexdigit+
      0 (o|O) octaldigit+
      0 (b|B) bitdigit+

sbyte : (int|xint) 'y'

byte  : (int|xint) 'uy'

int16 : (int|xint) 's'

uint16 : (int|xint) 'us'

int32 : (int|xint) 'l'

uint32 :
      (int|xint) 'ul'
      (int|xint) 'u'
```

```
nativeint : (int|xint) 'n'

unativeint : (int|xint) 'un'

int64 : (int|xint) 'L'

uint64 :
      (int|xint) 'UL'
      (int|xint) 'uL'

ieee32 :
      float [Ff]
      xint 'lf'

ieee64 :
      float
      xint 'LF'

bignum : int ('Q' | 'R' | 'Z' | 'I' | 'N' | 'G')

decimal : (float|int) [Mm]

float :
      digit+ . digit*
      digit+ (. digit* )? (e|E) (+|-)? digit+

reserved-literal-formats :
      (xint | ieee32 | ieee64) ident-char+
```

A.1.7 Line Directives

```
line-directive :
      # int
      # int string
      # int verbatim-string
      #line int
      #line int string
      #line int verbatim-string
```

A.1.8 Identifier Replacements

```
__SOURCE_DIRECTORY__
__SOURCE_FILE__
__LINE__
```

A.1.9 Operators

A.1.9.1 Operator Names

```
ident-or-op :
      ident
      ( op-name )
      (*)

op-name :
      symbolic-op
      range-op-name
      active-pattern-op-name
```

```
range-op-name :
      ..
      .. ..

active-pattern-op-name :
      | ident | ... | ident |
      | ident | ... | ident | _ |
```

A.1.9.2 Symbolic Operators

```
first-op-char : one of
       !%&*+-./<=>@^|~

op-char :
      first-op-char
      ?

quote-op-left :
      <@ <@@

quote-op-right :
      @> @@>

symbolic-op:
      ?
      ?<-
      first-op-char op-char*
      quote-op-left
      quote-op-right
```

A.1.9.3 Infix and Prefix Operators

The OP marker represents all *symbolic-op* tokens that begin with the indicated prefix, except for tokens that appear elsewhere in the table.

```
infix-or-prefix-op : one of
      +,  -, +., -., %, &, &&

prefix-op :
      infix-or-prefix-op
      ~ ~~ ~~~      (and any repetitions of ~)
      !OP           (all tokens that begin with ! except !=)

infix-op :
      infix-or-prefix-op
      -OP +OP || <OP >OP = |OP &OP ^OP *OP /OP %OP !=
                        (or any of these preceded by one or more '.')
      :=
      ::
      $
      or
      ?
```

A.1.9.4 Constants

```
const :
      sbyte
      int16
      int32
      int64
      byte
```

```
    uint16
    uint32
    int
    uint64
    ieee32
    ieee64
    bignum
    char
    string
    verbatim-string
    triple-quoted-string
    bytestring
    verbatim-bytearray
    bytechar
    false
    true
    ()
```

A.2 Syntactic Grammar

In general, this syntax summary describes full syntax. By default, however, `.fs`, `.fsi`, `.fsx`, and `.fsscript` files support lightweight syntax, in which indentation replaces **begin**/**end** and **done** tokens. This appendix uses **begin**$_{opt}$, **end**$_{opt}$, and **done**$_{opt}$ to indicate that these tokens are omitted in lightweight syntax. Complete rules for lightweight syntax appear in §15.1.

To disable lightweight syntax:

```
#indent "off"
```

When lightweight syntax is disabled, whitespace can include tab characters:

```
whitespace : [ ' ' '\t' ]+
```

A.2.1 Program Format

```
implementation-file :
    namespace-decl-group ... namespace-decl-group
    named-module
    anonynmous-module

script-file : implementation-file

signature-file:
    namespace-decl-group-signature ... namespace-decl-group-signature
    anonynmous-module-signature
    named-module-signature

named-module : module long-ident module-elems

anonymous-module : module-elems

named-module-signature : module long-ident module-signature-elements

anonymous-module-signature : module-signature-elements

script-fragment : module-elems
```

A.2.1.1 Namespaces and Modules

```
namespace-decl-group :
      namespace long-ident module-elems
      namespace global module-elems

module-defn : attributes_opt module access_opt ident = begin_opt module-defn-body end_op

module-defn-body : begin module-elems_opt end

module-elem :
      module-function-or-value-defn
      type-defns
      exception-defn
      module-defn
      module-abbrev
      import-decl
      compiler-directive-decl

module-function-or-value-defn :
      attributes_opt let function-defn
      attributes_opt let value-defn
      attributes_opt let rec_opt function-or-value-defns
      attributes_opt do expr

import-decl : open long-ident

module-abbrev : module ident = long-ident

compiler-directive-decl : # ident string ... string

module-elems : module-elem ... module-elem

access :
      private
      internal
      public
```

A.2.1.2 Namespace and Module Signatures

```
namespace-decl-group-signature : namespace long-ident module-signature-elements

module-signature : module ident = begin_opt module-signature-body end_opt

module-signature-element :
      val mutable_opt curried-sig
      val value-defn
      type type-signatures
      exception exception-signature
      module-signature
      module-abbrev
      import-decl

module-signature-elements :
      begin_opt module-signature-element ... module-signature-element end_opt

module-signature-body : begin module-signature-elements end

type-signature :
      abbrev-type-signature
      record-type-signature
      union-type-signature
```

```
      anon-type-signature
      class-type-signature
      struct-type-signature
      interface-type-signature
      enum-type-signature
      delegate-type-signature
      type-extension-signature

type-signatures : type-signature ... and ... type-signature

type-signature-element :
      attributes_opt access_opt new : uncurried-sig
      attributes_opt member access_opt member-sig
      attributes_opt abstract access_opt member-sig
      attributes_opt override member-sig
      attributes_opt default member-sig
      attributes_opt static member access_opt member-sig
      interface type

abbrev-type-signature : type-name '=' type

union-type-signature : type-name '=' union-type-cases type-extension-elements-signature_opt

record-type-signature :
      type-name '=' '{' record-fields '}' type-extension-elements-signature_opt

anon-type-signature : type-name '=' begin type-elements-signature end

class-type-signature : type-name '=' class type-elements-signature end

struct-type-signature : type-name '=' struct type-elements-signature end

interface-type-signature : type-name '=' interface type-elements-signature end

enum-type-signature : type-name '=' enum-type-cases

delegate-type-signature :  type-name '=' delegate-sig

type-extension-signature : type-name type-extension-elements-signature

type-extension-elements-signature : with type-elements-signature end
```

A.2.2 Types and Type Constraints

```
type :
      ( type )
      type -> type
      type * ... * type
      typar
      long-ident
      long-ident<type-args>
      long-ident< >
      type long-ident
      type[ , ... , ]
      type typar-defns
      typar :> type
      #type

type-args :=  type-arg, ..., type-arg
```

```
type-arg :=
    type
    measure
    static-parameter

atomic-type :
      type : one of
              #type typar ( type ) long-ident long-ident<types>)

typar :
      _
      'ident
      ^ident

constraint :
      typar :> type
      typar : null
      static-typars : (member-sig )
      typar : (new : unit -> 'T)
      typar : struct
      typar : not struct
      typar : enum<type>
      typar : unmanaged
      typar : delegate<type, type>

typar-defn : attributes_opt typar

typar-defns  : < typar-defn, ..., typar-defn typar-constraints_opt >

typar-constraints : when constraint and ... and constraint

static-typars :
      ^ident
      (^ident or ... or ^ident)
```

A.2.2.1 Equality and Comparison Constraints

```
typar : equality
typar : comparison
```

A.2.2.2 Type Providers

```
static-parameter =
    static-parameter-value
    id = static-parameter-value

static-parameter-value =
    const
    const expr
```

A.2.3 Expressions

```
expr :
      const
      ( expr )
```

```
    begin expr end
    long-ident-or-op
    expr '.' long-ident-or-op
    expr expr
    expr(expr)
    expr<types>
    expr infix-op expr
    prefix-op expr
    expr.[expr]
    expr.[slice-ranges]
    expr <- expr
    expr , ... , expr
    new type expr
    { new base-call object-members interface-impls }
    { field-initializers }
    { expr with field-initializers }
    [ expr ; ... ; expr ]
    [| expr ; ... ; expr |]
    expr { comp-or-range-expr }
    [ comp-or-range-expr]
    [| comp-or-range-expr |]
    lazy expr
    null
    expr : type
    expr :> type
    expr :? type
    expr :?> type
    upcast expr
    downcast expr
```

In the following four expression forms, the `in` token is optional if *expr* appears on a subsequent line and is aligned with the *let* token.

```
    let function-defn in expr
    let value-defn in expr
    let rec function-or-value-defns in expr
    use ident = expr in expr

    fun argument-pats -> expr
    function rules
    match expr with rules
    try expr with rules
    try expr finally expr
    if expr then expr elif-branches_opt else-branch_opt
    while expr do expr done_opt
    for ident = expr to expr do expr done_opt
    for pat in expr-or-range-expr do expr done_opt
    assert expr
    <@ expr @>
    <@@ expr @@>
    %expr
    %%expr
    (static-typars : (member-sig) expr)
    expr $app expr       // equivalent to "expr(expr)"
    expr $sep expr       // equivalent to "expr; expr"
    expr $tyapp < types > // equivalent to "expr<types>"

    expr< >

exprs : expr ',' ... ',' expr
```

```
expr-or-range-expr :
      expr
      range-expr

elif-branches : elif-branch ... elif-branch

elif-branch : elif expr then expr

else-branch : else expr

function-or-value-defn :
    function-defn
    value-defn

function-defn :
    inline_opt access_opt ident-or-op typar-defns_opt argument-pats return-type_opt = expr
value-defn :
    mutable_opt access_opt pat typar-defns_opt return-type_opt = expr

return-type :
      : type

function-or-value-defns :
      function-or-value-defn and ... and function-or-value-defn

argument-pats: atomic-pat ... atomic-pat

field-initializer : long-ident = expr

field- initializer s : field-
initializer ; ... ; field-initializer

object-construction :
      type expr
      type

base-call :
      object-construction
      object-construction as ident

interface-impls : interface-impl ... interface-impl

interface-impl : interface type object-members_opt

object-members : with member-defns end

member-defns :  member-defn ... member-defn
```

A.2.3.1 Computation and Range Expressions

```
comp-or-range-expr :
      comp-expr
      short-comp-expr
      range-expr
```

```
comp-expr :
      let! pat = expr in comp-expr
      let  pat = expr in comp-expr
      do! expr in comp-expr
      do  expr in comp-expr
      use! pat = expr in comp-expr
      use  pat = expr in comp-expr
      yield! expr
      yield  expr
      return! expr
      return  expr
      if expr then comp-expr
      if expr then comp-expr else comp-expr
      match expr with comp-rules
      try comp-expr with comp-rules
      try comp-expr finally expr
      while expr do expr done_opt
      for ident = expr to expr do comp-expr done_opt
      for pat in expr-or-range-expr do comp-expr done_opt
      comp-expr; comp-expr
      expr

comp-rule : pat pattern-guard_opt -> comp-expr

comp-rules : '|'_opt comp-rule '|' ... '|' comp-rule

short-comp-expr : for pat in expr-or-range-expr -> expr

range-expr :
      expr .. expr
      expr .. expr .. expr

slice-ranges : slice-range , … , slice-range

slice-range :
      expr
      expr..
      ..expr
      expr..expr
      '*'
```

A.2.3.2 Computation Expressions

```
expr { for ... }
expr { let ... }
expr { let! ... }
expr { use ... }
expr { while ... }
expr { yield ... }
expr { yield! ... }
expr { try ... }
expr { return ... }
expr { return! ... }
```

A.2.3.3 Sequence Expressions

```
seq { comp-expr }
seq { short-comp-expr }
```

A.2.3.4 Range Expressions

```
seq { e1 .. e2 }
seq { e1 .. e2 .. e3 }
```

A.2.3.5 Copy and Update Record Expression

```
{ expr with field-label1 = expr1 ; … ; field-labeln = exprn }
```

A.2.3.6 Dynamic Operator Expressions

```
expr ? ident                → (?) expr "ident"
expr1 ? (expr2)             → (?) expr1 expr2
expr1 ? ident <- expr2      → (?<-) expr1 "ident" expr2
expr1 ? (expr2) <- expr3    → (?<-) expr1 expr2 expr3
```

"*ident*" is a string literal that contains the text of *ident*.

A.2.3.7 AddressOf Operators

```
&expr
&&expr
```

A.2.3.8 Lookup Expressions

```
e1.[eargs]                          → e1.get_Item(eargs)
e1.[eargs] <- e3                    → e1.set_Item(eargs, e3)
```

A.2.3.9 Slice Expressions

```
e1.[sliceArg1, ,,, sliceArgN]                 → e1.GetSlice( args1,…,argsN)
e1.[sliceArg1, ,,, sliceArgN] <- expr → e1.SetSlice( args1,…,argsN, expr)
```

where each sliceArgN is a *slice-range* and translated to argsN (giving one or two args) as follows:

```
*       → None, None
e1..    → Some e1, None
..e2    → None, Some e2
e1..e2  → Some e1, Some e2
idx     → idx
```

A.2.3.10 Shortcut Operator Expressions

```
expr1 && expr2              → if expr1 then expr2 else false
expr1 || expr2              → if expr1 then true else expr2
```

A.2.3.11 Deterministic Disposal Expressions

```
use ident = expr1 in expr2
```

A.2.4 Patterns

```
rule : pat pattern-guardopt -> expr

pattern-guard : when expr

pat :
```

```
      const
      long-ident pat-param_opt pat_opt
      _
      pat as ident
      pat '|' pat
      pat '&' pat
      pat :: pat
      pat : type
      pat,...,pat
      (pat)
      list-pat
      array-pat
      record-pat
      :? atomic-type
      :? atomic-type as ident
      null
      attributes pat

list-pat :
      [ ]
      [ pat ; ... ; pat ]

array-pat :
      [| |]
      [| pat ; ... ; pat |]

record-pat : { field-pat ; ... ; field-pat }

atomic-pat :
    pat      one of
               const  long-ident  list-pat  record-pat  array-pat  (pat)
             :? atomic-type
                       null  _ _


field-pat : long-ident = pat

pat-param :
      const
      long-ident
      [ pat-param ; ... ; pat-param ]
      ( pat-param, ..., pat-param )
      long-ident pat-param
      pat-param : type
      <@ expr @>
      <@@ expr @@>
      null

pats :  pat , ... , pat

field-pats : field-pat ; ... ; field-pat

rules : '|'_opt rule '|' ... '|' rule
```

A.2.5 Type Definitions

```
type-defn :
      abbrev-type-defn
      record-type-defn
```

```
      union-type-defn
      anon-type-defn
      class-type-defn
      struct-type-defn
      interface-type-defn
      enum-type-defn
      delegate-type-defn
      type-extension

type-name : attributes_opt access_opt ident typar-defns_opt

abbrev-type-defn : type-name = type

union-type-defn : type-name '=' union-type-cases type-extension-elements_opt

union-type-cases : '|'opt union-type-case '|' ... '|' union-type-case

union-type-case : attributes_opt union-type-case-data

union-type-case-data :
      ident                  -- nullary union case
      ident of union-type-field * ... * union-type-field  -- n-ary union case
      ident : uncurried-sig  -- n-ary union case

union-type-field :
      type                   -- unnamed union type field
      ident : type           -- named union type field

anon-type-defn :
      type-name primary-constr-args_opt object-val_opt '=' begin class-type-body end

record-type-defn : type-name = '{' record-fields '}' type-extension-elements_opt

record-fields : record-field ; ... ; record-field ;_opt

record-field :  attributes_opt mutable_opt access_opt ident : type

class-type-defn :
      type-name primary-constr-args_opt object-val_opt '=' class class-type-body end

as-defn : as ident

class-type-body :
   begin_opt class-inherits-decl_opt class-function-or-value-defns_opt type-defn-elements_opt end_opt

class-inherits-decl : inherit type expr_opt

class-function-or-value-defn :
      attributes_opt static_opt let rec_opt function-or-value-defns
      attributes_opt static_opt do expr

struct-type-defn :
    type-name primary-constr-args_opt as-defn_opt '=' struct struct-type-body end

struct-type-body : type-defn-elements

interface-type-defn : type-name '=' interface interface-type-body end

interface-type-body : type-defn-elements
```

```
exception-defn :
      attributes_opt exception union-type-case-data
      attributes_opt exception ident = long-ident

enum-type-defn : type-name '=' enum-type-cases

enum-type-cases : '|'_opt enum-type-case '|' ... '|' enum-type-case

enum-type-case : ident '=' const

delegate-type-defn : type-name '=' delegate-sig

delegate-sig : delegate of uncurried-sig

type-extension : type-name type-extension-elements

type-extension-elements : with type-defn-elements end

type-defn-element :
      member-defn
      interface-impl
      interface-signature

type-defn-elements : type-defn-element ... type-defn-element

primary-constr-args : attributes_opt access_opt (simple-pat, ... , simplepat)

simple-pat :
    | ident
    | simple-pat : type



additional-constr-defn :
      attributes_opt access_opt new pat as-defn = additional-constr-expr

additional-constr-expr :
      stmt ';' additional-constr-expr
      additional-constr-expr then expr
      if expr then additional-constr-expr else additional-constr-expr
      let val-decls in  additional-constr-expr
      additional-constr-init-expr

additional-constr-init-expr :
      '{' class-inherits-decl field-initializers '}'
      new type expr

member-defn :
      attributes_opt static_opt member access_opt method-or-prop-defn
      attributes_opt abstract member_opt access_opt member-sig
      attributes_opt override access_opt method-or-prop-defn
      attributes_opt default access_opt method-or-prop-defn
      attributes_opt static_opt val mutable_opt access_opt ident : type
      additional-constr-defn

method-or-prop-defn :
      ident_opt function-defn
      ident_opt value-defn
      ident_opt ident with function-or-value-defns
      member ident = exp
```

```
      member ident = exp with get
      member ident = exp with set
      member ident = exp with get,set
      member ident = exp with set,get

member-sig :
      ident typar-defns_opt : curried-sig
      ident typar-defns_opt : curried-sig with get
      ident typar-defns_opt : curried-sig with set
      ident typar-defns_opt : curried-sig with get,set
      ident typar-defns_opt : curried-sig with set,get

curried-sig : args-spec -> ... -> args-spec -> type

uncurried-sig : args-spec -> type

args-spec : arg-spec * ... * arg-spec

arg-spec : attributes_opt arg-name-spec_opt type

arg-name-spec : ?_opt ident :

interface-spec : interface type
```

A.2.5.1 Property Members

```
static_opt member ident._opt ident = expr
static_opt member ident._opt ident with get pat = expr
static_opt member ident._opt ident with set pat_opt pat= expr
static_opt member ident._opt ident with get pat = expr and set pat_opt pat = expr
static_opt member ident._opt ident with set pat_opt pat = expr and get pat = expr
```

A.2.5.2 Method Members

```
static_opt member ident._opt ident pat1 ... patn = expr
```

A.2.5.3 Abstract Members

```
abstract access_opt member-sig

member-sig :
    ident typar-defns_opt : curried-sig
    ident typar-defns_opt : curried-sig with get
    ident typar-defns_opt : curried-sig with set
    ident typar-defns_opt : curried-sig with get, set
    ident typar-defns_opt : curried-sig with set, get

curried-sig : args-spec_1 -> ... -> args-spec_n -> type
```

A.2.5.4 Implementation Members

```
override ident.ident pat1 ... patn = expr
default ident.ident pat1 ... patn = expr
```

A.2.6 Units Of Measure

```
measure-literal-atom :
      long-ident
      ( measure-literal-simp )
```

```
measure-literal-power :
      measure-literal-atom
      measure-literal-atom ^ int32

measure-literal-seq :
      measure-literal-power
      measure-literal-power measure-literal-seq

measure-literal-simp :
      measure-literal-seq
      measure-literal-simp * measure-literal-simp
      measure-literal-simp / measure-literal-simp
      / measure-literal-simp
      1

measure-literal :
      _
      measure-literal-simp

const :
      ...
      sbyte < measure-literal >
      int16 < measure-literal >
      int32 < measure-literal >
      int64 < measure-literal >
      ieee32 < measure-literal >
      ieee64 < measure-literal >
      decimal < measure-literal >

measure-atom :
      typar
      long-ident
      ( measure-simp )

measure-power :
      measure-atom
      measure-atom ^ int32

measure-seq :
      measure-power
      measure-power measure-seq

measure-simp :
      measure-seq
      measure-simp * measure-simp
      measure-simp / measure-simp
      / measure-simp
      1

measure :
      _
     measure-simp
```

A.2.7 Custom Attributes and Reflection

```
attribute : attribute-target:opt object-construction

attribute-set : [< attribute ; ... ; attribute >]
```

```
attributes : attribute-set ... attribute-set

attribute-target :
      assembly
      module
      return
      field
      property
      param
      type
      constructor
      event
```

A.2.8 Compiler Directives

Compiler directives in non-nested modules or namespace declaration groups:

```
# id string ... string
```

A.3 ML Compatibility Features

A.3.1 Conditional Compilation

```
start-fsharp-token :
      "(*IF-FSHARP"
      "(*F#"
end-fsharp-token :
      "ENDIF-FSHARP*)"
      "F#*)"
start-ml-token : "(*IF-OCAML*)"
end-ml-token : "(*ENDIF-OCAML*)"
```

A.3.2 Extra Syntactic Forms

```
ocaml-ident-keyword : one of
      asr land lor lsl lsr lxor mod

expr :
      ...
      expr.(expr)           // array lookup
      expr.(expr) <- expr   // array assignment

type :
      ...
      (type,...,type) long-ident // generic type instantiation

module-implementation :
      ...
      module ident = struct ... end

module-signature :
      ...
      module ident : sig ... end
```

A.3.3 Extra Operators

e_1 `or` e_2 → `(or)` e_1 e_2
e_1 `&` e_2 → `(&)` e_1 e_2